Building and Attacking an ICS/SCADA Honeypot

A Conpot decoy PLC over Modbus, SNMP, S7comm, FTP and BACnet — live attack probes, automated log analysis and a real-time dashboard

Khan Imaad Ali (22410104)

Makrand Gaikwad (22410110)

[Course / Module — University]

October 26, 2026

Outline

Background

Architecture

Attacking the honeypot

Results

Log analysis & detection

Wrap-up

PART 1

Background

Why an ICS/SCADA honeypot?

- Industrial Control Systems (ICS/SCADA) run power, water and factory plants using legacy protocols — **Modbus**, **S7comm**, **SNMP** — built with *little or no authentication*.
- These devices are increasingly reachable from the network, and attackers actively scan for them (Shodan, automated ICS worms).
- A **honeypot** is a decoy device: it looks like a real PLC, has no real process behind it, and *logs everything an attacker does*.

Goal

Stand up a realistic ICS honeypot, attack it over **five protocols**, and turn the raw honeypot log into structured events, risk scores and a live dashboard.

Conpot — the honeypot engine

- Open-source, low-interaction ICS honeypot from the **Honeynet Project**.
- Emulates a **Siemens SIMATIC S7-200** PLC.
- Speaks **Modbus/TCP**, **S7comm**, **SNMP**, **HTTP** — plus **FTP**, **BACnet** and more — in one container.
- Answers with a believable device identity, so a scanner thinks it found a real PLC — then records every interaction.

Two halves of the system

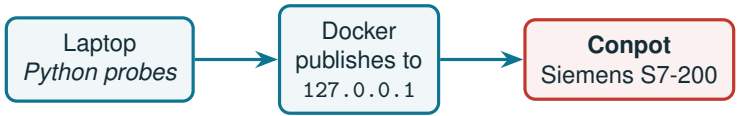
Attacker side — deploy Conpot, probe it, capture `conpot.log`.

Analysis side — parse the log, detect attack patterns, score risk, build a dashboard.

PART 2

Architecture

System architecture



The Conpot **Docker** container (built from the project's custom image) publishes its ICS ports on `127.0.0.1`, so every probe targets localhost and reaches the container directly — no separate machine needed.

Service	Laptop port	Conpot port
Modbus	502	5020
S7comm	102	10201
SNMP (udp)	161	16100
HTTP	8080	8800
FTP	2121	2121
BACnet (udp)	47808	47808

PART 3

Attacking the honeypot

Attack methodology — five probes

Five Python probes run from the laptop against `127.0.0.1`, each generating clean, labelled traffic — the classic **scan** → **read** → **write** chain.

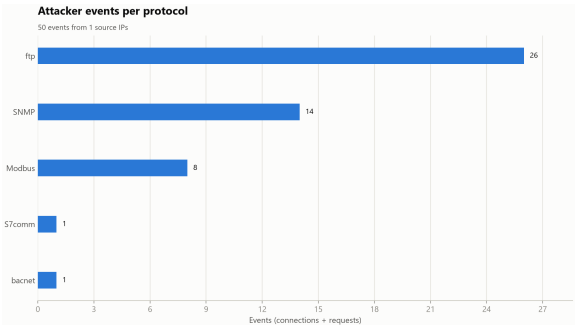
- **Modbus** (pymodbus) — read coils, discrete inputs and registers, then **write** a holding register (value 1234), *accepted* by the custom template: a live tamper.
- **SNMP** (pysnmp) — `GET` system OIDs and `WALK` the system subtree; returns the Siemens identity.
- **S7comm** (snap7) — establish an S7 session and request CPU identification (SZL) — what PLC-scanning tools do first.
- **FTP** (ftplib) — anonymous control session (`SYST`/`PWD`/`STAT`/`HELP`).
- **BACnet** (socket) — a broadcast `Who-Is`, the building-automation reconnaissance move.

The write + multi-protocol scan is deliberate: it is a real attacker moving from *reconnaissance* to *tampering*.

PART 4

Results

What the honeypot captured



- **50 attacker events** from a single source (`172.17.0.1`, the Docker gateway — i.e. our laptop).
- **FTP** (26) — the control-session commands.
- **SNMP** (14) — the full system-tree walk.
- **Modbus** (8) — register reads and an accepted write.
- **S7comm** (1), **BACnet** (1) — reconnaissance contacts.

The deception works — twice

SNMP: a convincing identity

sysDescr	Siemens, SIMATIC, S7-200
sysName	CP 443-1 EX40
sysContact	Siemens AG
sysLocation	Venus

A scanner now believes this is a real Siemens PLC.

Modbus: real data & a live tamper

Custom template — slave 1 answers with realistic process data:

holding registers = `[1480, 1465, 85, ...]` (pump setpoints)

input registers = `[412, 538, 1260, ...]` (sensors)

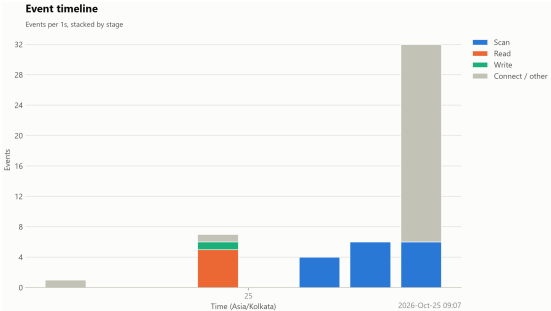
A write of **1234** to holding register 0 is **accepted** — read-back = 1234.

An attacker can change the PLC's "process" — the core ICS threat.

PART 5

Log analysis & detection

From raw log to structured events

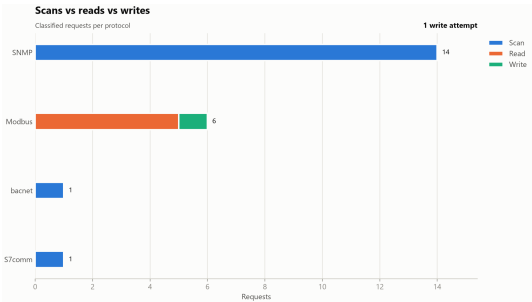


Parser. `conpot.log` → a pandas table, one row per event with:

- protocol, request type, function code
- action: **scan** / **read** / **write**
- source IP (+ private / Docker-gateway flags)

147 log lines → 50 clean events, verified by a **130+ test** suite with CI.

Attack-sequence detection & risk scoring



Per source IP, the detector scores risk from the attack *sequence*, multi-protocol scanning and any write attempts.

Our capture: `172.17.0.1`

Risk: **High**
Sequence: **scan** → **read** → **write**
Protocols: modbus, s7comm, snmp, ftp, bacnet (5)
Write attempt flagged

The scan → read → write chain across five protocols is the classic recon-to-tamper pattern.

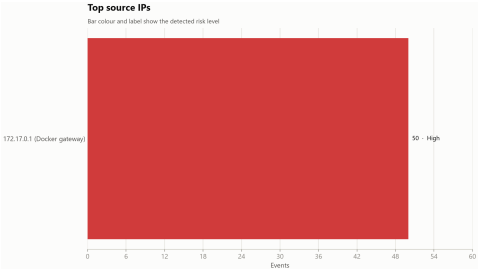
Live dashboard & attack simulator

A **Streamlit** dashboard reads the log live and shows:

- a threat-level **posture banner** + KPI tiles,
- interactive charts and a colour-coded risk-by-IP table,
- a **“Live from Docker”** source — reads the running container directly.

Attack Simulator tab: run the attack *from the UI* — the scan → read → write chain lights up with a PLC-status card and an attacker → honeypot view.

Run with: `streamlit run dashboard/app.py`



Top source IPs, coloured by risk

PART 6

Wrap-up

Ethics & scope

- Every probe targets **our own** honeypot on **localhost** — a private, isolated setup; the attack console refuses any non-local target.
- Conpot is purpose-built to be scanned and attacked; no third party is involved.
- No tool here is pointed at any host we do not own or have permission to test.

Why this is safe and useful

The honeypot has no real process to disrupt; the value is entirely in *observing and analysing* attacker behaviour in a controlled setting.

Conclusion & future work

What was built

- A Conpot ICS honeypot (Siemens S7-200) with a **custom template** that returns realistic Modbus data and *accepts writes*.
- Attack traffic over **five protocols** (Modbus, SNMP, S7comm, FTP, BACnet); 50 captured events scored **High** risk.
- An automated pipeline — parser, risk detection, interactive charts and a live dashboard with an **Attack Simulator** — backed by a 130+ test suite and CI.

Future work

- Expose the honeypot on an isolated public-facing network to capture *real* internet scans.
- Classify more protocols (TFTP, EtherNet/IP); add per-protocol attack animation and richer register realism.

Thank you

Questions?

Conpot ICS/SCADA honeypot — attack & analysis project